CPIT 305- Java Exception Handling

In Java, exception handling is a mechanism to handle runtime errors, allowing the normal flow of a program to continue. Exceptions are events that occur during program execution that disrupt the normal flow of instructions.

Basic try-catch Example

- The try block contains code that might throw an exception,
- The catch block handles the exception if it occurs.

```
    class Geeks{
     public static void main(String[] args) {

         int n = 10;
         int m = 0;

         try {
             int ans = n / m;
             System.out.println("Answer: " + ans);
         } catch (ArithmeticException e){
             System.out.println("Error: Division by 0!");
         }
     }
}
```

Output

```
Error: Division by 0!
```

Finally Block

The finally block always executed whether an exception is thrown or not. The `finally` is used for closing resources like db connections, open files and network connections, It is used after a try-catch block to execute code that must run.

```
class FinallyExample {
```

```
    public static void main(String[] args){

        int[] numbers = { 1, 2, 3 };
        try {
            // This will throw ArrayIndexOutOfBoundsException
            System.out.println(numbers[5]);

        }
        catch (ArrayIndexOutOfBoundsException e){

            System.out.println("Exception caught: " + e);
        }
        finally{
            System.out.println("This block always executes.");
        }
        System.out.println("Program continues...");
    }
}
```

Output

```
Exception caught: java.lang.ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 3

This block always executes.

Program continues...
```

throw and throws Keywords

1. throw: Used to explicitly throw a single exception. We use `throw` when something goes wrong (or “shouldn’t happen”) and we want to stop normal flow and hand control to exception handling.

```
class Demo {
    static void checkAge(int age) {

        if (age < 18) {
            throw new ArithmeticException("Age must be 18 or
above");
        }
    }

    public static void main(String[] args) {

        checkAge(15);
    }
}
```

Output:

Exception in thread "main" java.lang.ArithmeticException: Age must be 18 or above
at Demo.checkAge(Demo.java:5)
at Demo.main(Demo.java:11)

2. throws: Declares exceptions that a method might throw, informing the caller to handle them. It is mainly used with checked exceptions (explained below). If a method calls another method that throws a checked exception, and it doesn't catch it, it must declare that exception in its `throws` clause

```
import java.io.*;

class Demo {
    static void readFile(String fileName) throws IOException {

        FileReader file = new FileReader(fileName);
    }

    public static void main(String[] args){

        try {
            readFile("test.txt");
        } catch (IOException e){

            System.out.println("File not found: " +
e.getMessage());
        }
    }
}
```

Output

```
File not found: test.txt (No such file or directory)
```

Internal Working of try-catch Block:

- JVM executes code inside the `try` block.
- If an exception occurs, remaining `try` code is skipped and JVM searches for a matching `catch` block.
- If found, the `catch` block executes.
- Control then moves to the `finally` block (if present).
- If no matching `catch` is found, the exception is handled by JVM's default handler.
- The `finally` block always executes, whether an exception occurs or not.

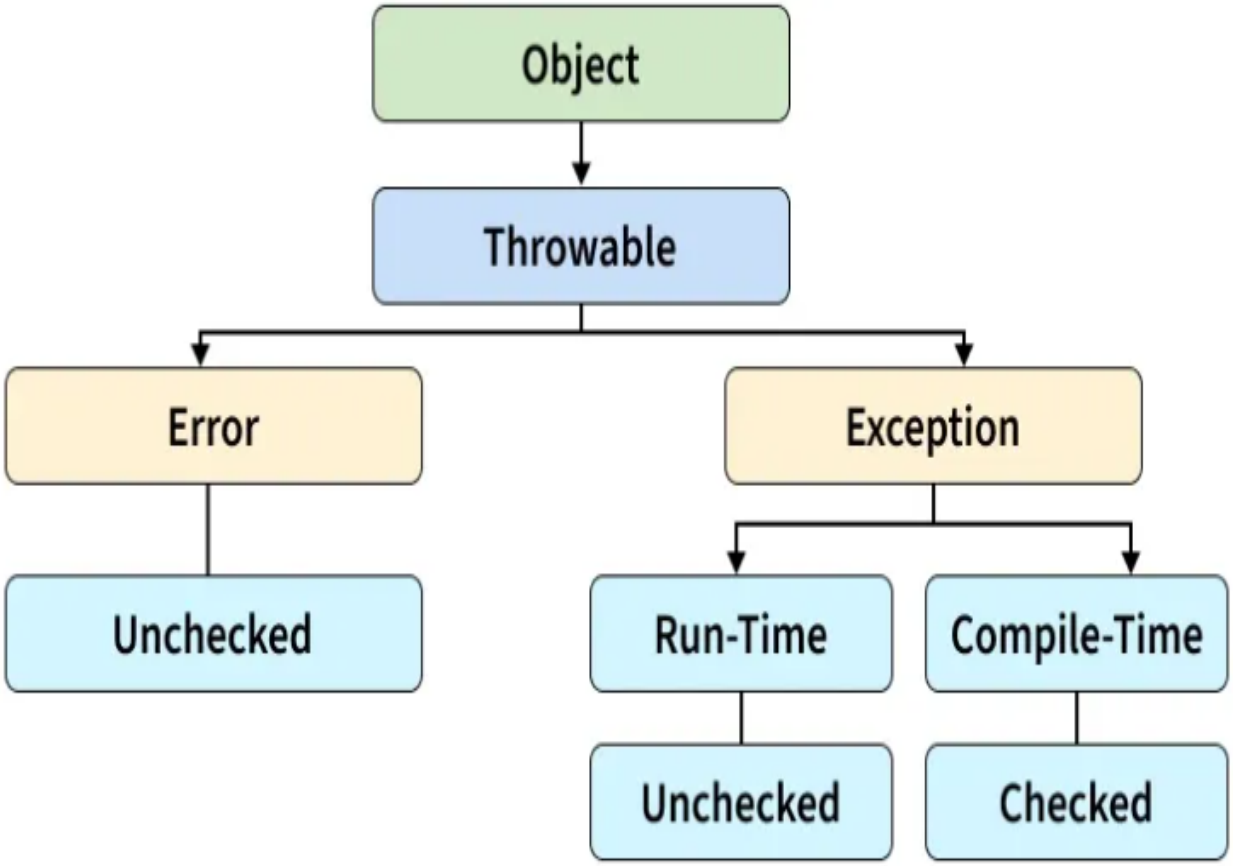
Note: *When an exception occurs and is not handled, the program terminates abruptly and the code after it, will never execute.*

Java Exception Hierarchy

In Java, all exceptions and errors are subclasses of the Throwable class. It has two main branches

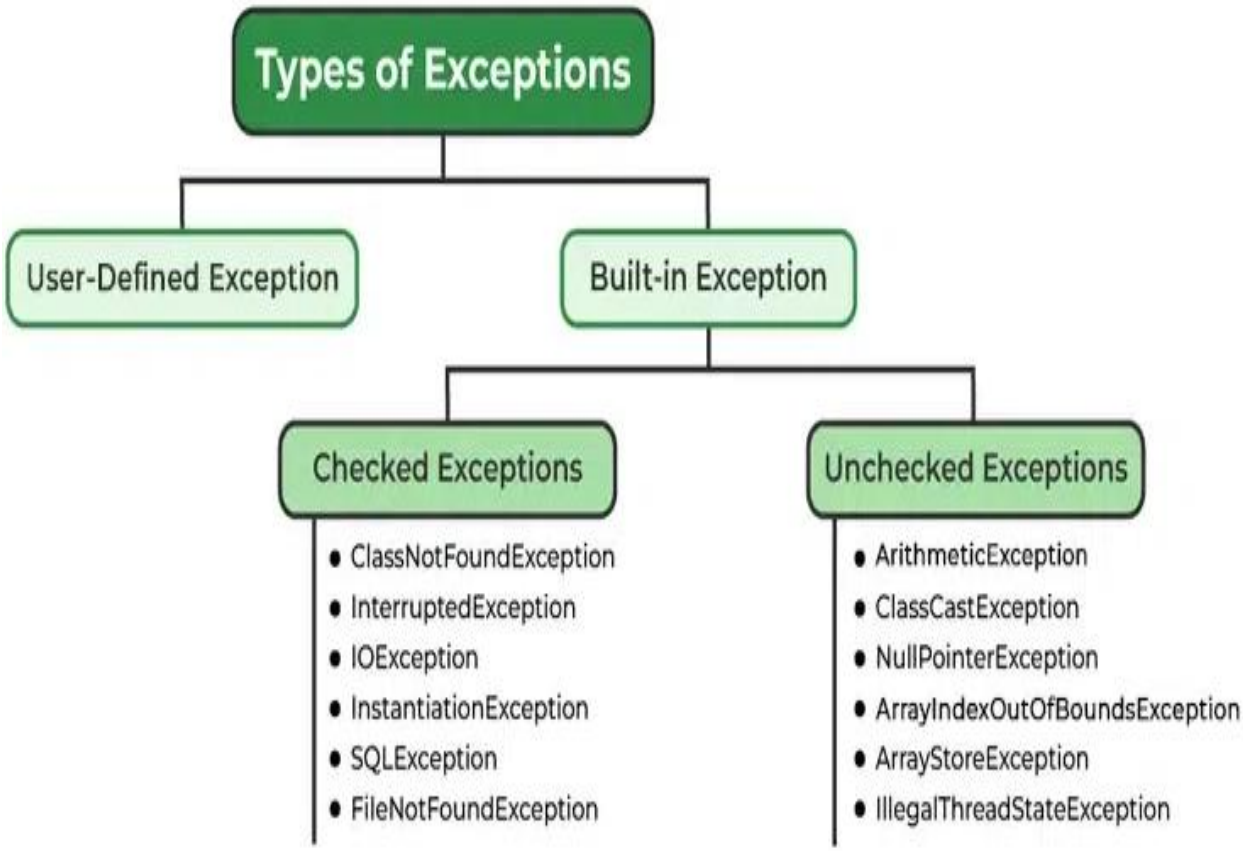
1. Exception.
2. Error

The below figure demonstrates the exception hierarchy in Java:



Types of Java Exceptions

Java defines several types of exceptions that relate to its various class libraries. Java also allows users to define their it's exceptions.



Exception

1. Built-in Exception

Built-in Exception are pre-defined exception classes provided by Java to handle common errors during program execution. There are two type of built-in exception in java.

- **Checked Exception:** These exceptions are checked at compile time, forcing the programmer to handle them explicitly.
- **Unchecked Exception:** These exceptions are checked at runtime and do not require explicit handling at compile time.

To know more about Checked and Unchecked Exception -> Checked and Unchecked Exception

2. User-Defined Exception

Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called "user-defined Exceptions".

Methods to Print the Exception Information

- **printStackTrace()**: Prints the full stack trace of the exception, including the name, message and location of the error.
- **toString()**: Prints exception information in the format of the Name of the exception.
- **getMessage()** : Prints the description of the exception

Nested try-catch

In Java, you can place one try-catch block inside another to handle exceptions at multiple levels.

```
public class NestedTryExample {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block");
            try {
                int a = 10 / 0; // This causes
ArithmeticException
            } catch (ArithmeticException e) {
                System.out.println("Inner catch: " + e);
            }
            String str = null;
            System.out.println(str.length()); // This causes
NullPointerException
        } catch (NullPointerException e) {
            System.out.println("Outer catch: " + e);
        }
    }
}
```

Output

```
Outer try block

Inner catch: java.lang.ArithmeticException: / by zero

Outer catch: java.lang.NullPointerException: Cannot invoke
"String.length()" because "<local1>" is null
```

Handling Multiple Exception

We can handle multiple type of exceptions in Java by using multiple catch blocks, each catching a different type of exception.

```
try {

    // Code that may throw an exception

} catch (ArithmeticException e) {

    // Code to handle the exception

} catch(ArrayIndexOutOfBoundsException e){

    // Code to handle the another exception

}catch(NumberFormatException e){

     // Code to handle the another exception
}
```

How Does JVM Handle an Exception?

When an Exception occurs, the JVM creates an exception object containing the error name, description and program state. Creating the exception object and handling, it in the run-time system is called throwing an exception. There might be a list of the methods that had been called to get to the method where an exception occurred. This ordered list of methods is called call stack. Now the following procedure will happen:

- The run-time system searches the call stack for an exception handler
- It starts searching from the method where the exception occurred and proceeds backward through the call stack.
- If a handler is found, the exception is passed to it.
- If no handler is found, the default exception handler terminates the program and prints the stack trace.

Exception in thread "abc" Name of Exception : Description
// Call Stack

Look at the below diagram to understand the flow of the call stack:

Illustration:

```
class Geeks{
```

```java
    public static void main(String args[])
    {
        // Taking an empty string
        String s = null;

        // Getting length of a string
        System.out.println(s.length());
    }
}
```

Try it on GfG Practice

Output:

```
PS C:\Devanshu> javac Geeks.java
PS C:\Devanshu> java Geeks
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String.length()"
 because "<local1>" is null
        at Geeks.main(Geeks.java:9)
PS C:\Devanshu>
```

output

Difference Between Exception and Error

Feature	Exception	Error
Definition	An event that occurs during program execution, disrupting normal flow, which can be handled using try-catch.	A serious problem that occurs in the JVM, generally cannot be handled by the application.
Package	java.lang.Exception	java.lang.Error
Recoverable	Yes, can be caught and handled.	No, usually not recoverable.
Examples	IOException, SQLException, ArithmeticException	OutOfMemoryError, StackOverflowError

Questions:

1) What is the purpose of Exception Handling in Java?

- A -To catch errors in the program
- B- To handle runtime errors, ensuring the program continues
- C- To avoid compilation errors
- D- To increase program speed

2) What will be the output for the following code?
Java public class ExceptionTest {
public static void main(String[] args) {
try { int result = 10 / 0;
} catch (ArithmeticException e)
{ System.out.println("Arithmetic Exception caught");
} catch (Exception e)
{ System.out.println("General Exception caught");
} finally
{ System.out.println("Finally block executed"); } } }

- A- Arithmetic Exception caught
- B-- Arithmetic Exception caughtFinally block executed
- C- General Exception caught
- D- Compilation Error

3. Which of the following best describes the difference between throw and throws in Java?

- A- throw is used for checked exceptions, throws is for unchecked
- B- throw declares an exception, throws handles it
- C- throw is used to explicitly throw an exception, throws is used in method declarations
- D- throw can be used only in catch, throws only in constructors